```
  let count = 0;
  return function inner() {
    count++;
    return count;
  };
}
const counter = outer();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Analysis: This question assesses how closures allow functions to retain access to variables in their outer lexical scope even after the outer function has finished executing. It also evaluates awareness of memory implications, since closures preserve references to variables, which can lead to memory retention if not managed carefully.

Answer: This example demonstrates how JavaScript closures preserve access to variables in their lexical scope. When `createCounter()` is invoked, it creates a new execution context where `count` is initialized to 0. It then returns an inner function that closes over the outer lexical environment, maintaining access to `count` even after `createCounter()` has finished executing. Because the returned function retains a live reference to the `count` variable via the closure, each call to `counter()` accesses and updates the same `count` value. This persistence across multiple invocations occurs because the scope chain allows the inner function to resolve `count` in the lexical scope of `createCounter()` rather than its own local scope. However, if the closure is kept alive unnecessarily or attached to objects that persist for the application's lifetime, the `count` variable will also remain in memory, potentially leading to a memory leak if not managed properly.

For this topic, are you able to answer these follow-up/probing questions?

- "What are the potential memory implications of using closures, and how can memory leaks be avoided?"
- "How does JavaScript’s garbage collector determine when to reclaim the memory used by count in this closure?"
- "Can you provide an example where closures are used to encapsulate private state in a module or class? What are the trade-offs compared to class fields?"